Final Project

Team: Single contributor, responsible for all aspects of the project, including data preprocessing, feature engineering, model selection (Logistic Regression), implementation, and evaluation. Will also document the entire process and provide insights into the results.

Context: League of Legends (LoL) is a team-based multiplayer online battle arena (MOBA) game developed by Riot Games. Players assume the role of unique champions with distinct abilities, teaming up in 5v5 matches. The primary objective of the game is to destroy the enemy's Nexus, located at the heart of their base, while navigating lanes, defeating enemy champions, and working with teammates to secure advantages like neutral objectives [1].

A) The problem identified is determining the earliest time frame during a League of Legends match where accurate predictions about the game's outcome (win or loss) can be made. Additionally, the project aims to analyze feature importance to identify which in-game metrics most significantly influence the match's result.

B) This problem is challenging due to the game's complexity, influenced by dynamic factors, high feature dimensionality, player skill variability, team strategies, and match dynamics.

C) Logistic Regression can be used to model the relationships between in-game metrics and match outcomes. By training the model on historical data, it can learn patterns and weights for features that contribute most to a team's likelihood of winning.

D) The project will develop a logistic regression model that accurately classifies the match outcome based on early-game data and identify the minimum time frame where predictions become reliable. The project will also determine which features most significantly influence winning.

E) Developing an early-prediction model is especially valuable for League of Legends players, as it helps them refine strategies by identifying the most impactful early-game metrics and recognizing when to abandon a game. The model can also provide developers with insights into key factors influencing match outcomes which can guide game balancing decisions.

Dataset: The dataset for this project is available on Kaggle: League of Legends SoloQ Ranked Games Dataset [2]. It contains data from ranked solo queue matches in the Platinum ELO tier of League of Legends from 2020. The dataset includes approximately 240,000 game frames across 25,000 matches. Game frames start at 10 minutes and are collected every two minutes until the end of the game. The dataset was collected directly from Riot API via scrapping and API calls. There are a total of 55 features for the blue team, including metrics such as kills, deaths, assists, gold difference, and objectives taken. The target variable *hasWon* is the binary value indicating whether the blue team won the match. Some time frames will be combined to reduced computational complexity while all relevant features will be included to ensure accurate predictions and meaningful feature importance analysis.

Proposed Solution:

A) This is a predictive problem, aiming to classify the match outcome based on early-game data at different time frames.

B) The target variable is *hasWon*. Feature variables include difference in gold, experience, champion level and vision score as well as cumulative kills, deaths and assists across the team. Control of neutral jungle objectives and towers are also included as binary values. The features are quite comprehensive and provide detailed insights into the game progress from the blue team's perspective.

C) Logistic Regression is chosen because it is well-suited for predicting binary outcomes like win or loss which align perfectly with the project's goals. Additionally, Logistic Regression provides interpretable coefficients that show how each feature impacts the probability of a win. This makes it easy to analyze feature importance and draw actionable insights. Logistic Regression is also more appropriate for the large dataset utilized in this project (~240,000 samples).

D) Data preprocessing involves handling missing values by dropping rows with missing data points and combining specific metrics. For example, instead of having individual feature variables for control of the nexus turret in each of the three lanes, it is computationally more efficient to combine them into one variable without losing much information. Time frames can also be combined into larger intervals since for most features, change isn't drastic between the two-minute time frames. K-fold Cross Validation and L1 Regularization will also be used. The final model will be evaluated based on the confusion matric and classification report as well as area under the ROC curve (AUC). These metrics are important for assessing the model's accuracy, recall and precision. Feature analysis can be done through analyzing coefficient magnitudes and utilizing the Odds Ratio.

E) Many previous studies on League of Legends analytics focus on full-game analysis or pre-game predictions [3][4]. The most similar study was conducted by Silva et al. using a simple Recurrent Neural Network to predict the results of professional LoL matches at 5-minute intervals [5]. Hodge et al. also utilized Logistic Regression algorithms during real time analysis of a similar MOBA game – DOTA 2 [6]. This project is unique in its emphasis on early-game prediction by finding the minimal time frame for reliable results and producing interpretable model outputs.

F) Libraries used will include numpy, pandas, matplotlib, seaborn and sklearn. Scikit-learn can also be used to make modeling easier.

Preprocessing:

- **i)** An 80-20 split was used to separate the data into training and testing sets. Additionally, 5-fold cross-validation was applied, where 80% of the data was used for training and 20% for validation within each fold.
- **ii)** Irrelevant features, such as *gameId* and *gameDuration*, were dropped as they did not contribute to predicting the game outcome. The *frame* column was utilized to segment the data into meaningful groups for modeling based on two-minute intervals. To address the issue of sparsity in data, such frames (e.g., 30 minutes or more), were combined into a single group. This decision was made to ensure sufficient data points for modeling while maintaining the focus on early game time predictions, which is the primary goal of this project.
- **iii)** To assess feature relationships and ensure no multicollinearity, a feature correlation heatmap was generated. This visualized the pairwise correlations between numerical features. Minimal correlations were observed between features, indicating no immediate need for feature removal (Figure 1. In Appendix).
- **iv)** Given the large dataset size and balanced sample distribution, standard data augmentation techniques were not required. The dataset provides sufficient variety and representation.
- **v)** All numerical features were standardized using StandardScaler to normalize the data, improving model convergence during training. Since the dataset contained no categorical features, one-hot encoding was not required.

Model Specifications: The logistic regression model was implemented as a neural network for binary classification to predict the target variable (*hasWon*). The model consisted of a single input layer with the number of features determined by the frame group and a single output layer with a sigmoid activation function. No hidden layers were used, keeping the model simple and interpretable.

Key hyperparameters included a learning rate of 0.01, and L1 regularization was applied with a penalty parameter ($\lambda = 0.01$). The binary cross-entropy loss function was used to minimize classification error, and the weights were optimized using the Adam optimizer. The training process involved 10 epochs, using full-batch gradient descent, where the entire dataset was used in each batch for parameter updates. 5-fold cross-validation was performed, where 80% of the data was used for training and 20% for validation in each fold. Metrics such as accuracy, F1-score, and AUC-ROC were computed for each fold to assess the model's effectiveness.

Training and validation loss were calculated after each of the 10 epochs for each frame group, which help identify convergence and potential over/underfitting.

Additionally, feature analysis was conducted using the coefficients of the logistic regression model. The coefficients were transformed into odds ratios to interpret the importance of each feature for each time frame, which is saved into a csv file.

Preliminary Results: The evaluation results for each frame group demonstrate the performance of the logistic regression model across different time segments. For frame group 10, the model achieved an accuracy of 70% with an AUC-ROC of 0.77, indicating moderate discrimination capability. Each frame group showed a slight improvement. In frame group 22, the model achieved its highest performance with an accuracy of 81% and an AUC-ROC of 0.89. The accuracy remained consistent in frame 24 and 26 at 80% but then dropped for later frame groups.

For every time frame, the graph of the validation and training loss shows a steady decline in both training and validation losses, demonstrating that the model is learning from the data and converging. This provides confidence in the training process. The plot for each time frame is saved as a PNG file.

Preprocessing Improvements: Removing the *expDiff* feature did improve performance by reducing noise and preventing the model from overfitting to less relevant information. This improvement is primarily because there exists a correlation between the features *champLevelDiff* and *expDiff* thus making the former redundant, particularly in early frames.

Selecting features based on high correlation with the target variable and switching the preprocessing technique from StandardScaler to MinMaxScaler did not enhance the model's performance. Compared to late game, in the earlier frames many features are not yet relevant and adding another selection criterion further restricts the already limited feature set. Additionally, the MinMaxScaler compresses feature values into a narrow range [0, 1], which likely diminished the model's ability to capture meaningful differences in features with diverse distributions or outliers [7]. The StandardScaler proved to be more robust and better suited for this dataset.

Model Specification Improvements: The number of training epochs was increased from 10 to 20, giving the model additional time to converge and achieve better optimization. This adjustment resulted in improved performance metrics, as observed in the loss curves, which showed convergence without signs of overfitting. By providing the model more training iterations, the overall accuracy and AUC-ROC scores improved, validating the effectiveness of this change.

The model was modified from a simple logistic regression setup into a deeper neural network to better capture non-linear relationships in the data. Three hidden layers were added with sizes [128, 64, 32], each followed by ReLU activation functions to introduce non-linearity [8]. This increased network depth allowed the model to learn more complex feature interactions, enhancing its capacity to model patterns in the data. The output layer remained unchanged as a single neuron with a sigmoid activation function, suitable for binary classification as in the case of game outcomes.

Various learning rates and batch sizes were explored to optimize the model's performance. Learning rates such as 0.1 and 0.001 were tested, but they either slowed down convergence significantly due to the large size of the dataset or led to suboptimal results compared to the

selected learning rate of 0.01. Similarly, smaller batch sizes were explored but did not provide significant improvements in performance.

Evaluation: The evaluation of the model involved several steps (As seen from Figure 2 in Appendix). 5-fold cross-validation was performed for each frame group, this allows the model to train on different subsets of the data while ensuring that the evaluation was not biased toward a specific subset. Key performance metrics such as accuracy, precision, recall, F1-score, and AUC-ROC were computed to assess the model's effectiveness in predicting game outcomes [9]. Feature importance was analyzed using permutation importance, rather than the previous feature coefficients, to better suite the non-linear model [10]. The top three influential features for each frame group are highlighted, providing insights into the model's decision-making process. These insights are also interpretable and actionable increasing the model's practical utility. Additionally, the model's performance was compared across various frame groups, showing how its predictive ability evolved as the game progressed and more data is accrued.

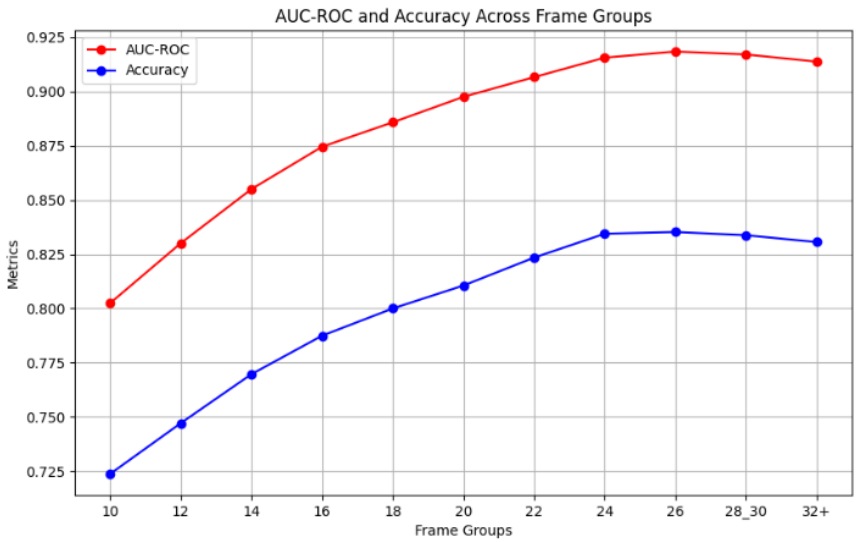


Figure 1. AUC-ROC and Accuracy across frame groups

In early frames (e.g., 10 and 12), the model still struggles to achieve high precision and recall. The confusion matrices also indicate higher false positive and false negative rates. Accuracy and AUC-ROC steadily increase from frame group 10, which has an accuracy of 72% and AUC-ROC of 0.80. Compared to preliminary results, model performance improved for all frames. After frame group 24, the performance stabilizes at high values, with accuracy around 83% and AUC-ROC between 0.91 and 0.92. As seen in Figure 1, this improvement in AUC-ROC reflects the model's increasing ability to distinguish between wins and losses as more data becomes available. Feature importance analysis further revealed that *champLevelDiff* and *goldDiff* are consistently the most important predictors across all frame groups, indicating their strong correlation with the target variable (*hasWon*). In later frame groups, features such as deaths,

turret destruction, and securing neutral objectives gain prominence, highlighting the increasing importance of late-game events in determining match outcomes. These results align with the intuitive understanding that early game predictions are more difficult.

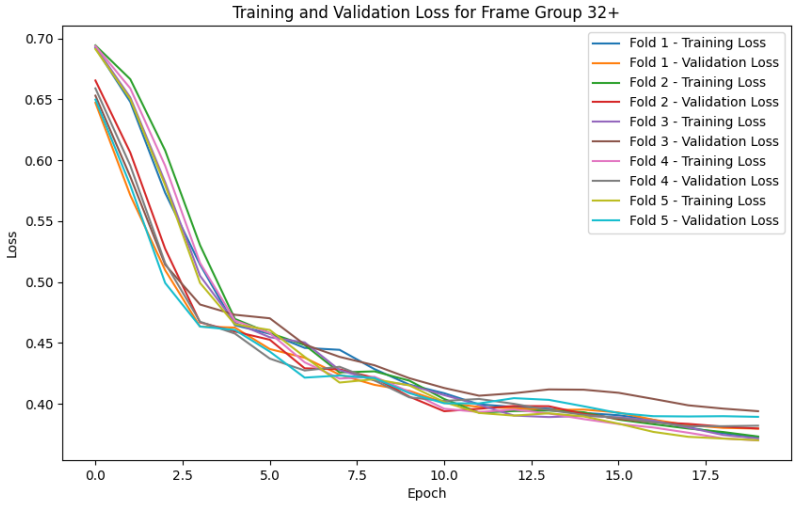


Figure 2. Training and Validation loss across each fold for frame group 32+

The training and validation losses were also monitored throughout the training process to evaluate the model's learning and detect potential overfitting or underfitting. Both training and validation loss curves consistently showed convergence over the 20 epochs across all frame groups, indicating effective learning.

Limitations: The primary limitation of the ML system is its reduced performance in early frame groups, where limited data and less meaningful features hinder accurate predictions. Additionally, the model relies on specific features from the game League of League Legends, which make it less generalizable to other contexts without significant retraining. Finally, as a stochastic system, it cannot achieve perfect predictions due to inherent variability and noise in the data.

REFERENCES

[1] R. Claytor and E. Ehrlich, “Riot Games and AWS bring esports ‘win probability’ stat to 2023 league of legends World Championships broadcasts | Amazon Web Services,” AWS Amazon, https://aws.amazon.com/blogs/gametech/riot-games-and-aws-bring-esports-win-probability-stat-to-2023-league-of-legends-world-championships-broadcasts/ (accessed Nov. 17, 2024).

[2] Y. L. Ma, “League of legends solo-Q ranked games,” Kaggle, https://www.kaggle.com/datasets/bobbyscience/league-of-legends-soloq-ranked-games (accessed Nov. 19, 2024).

[3] L. M. Costa, R. G. Mantovani, F. C. Monteiro Souza, and G. Xexeo, “Feature analysis to league of legends victory prediction on the picks and bans phase,” 2021 IEEE Conference on Games (CoG), pp. 01–05, Aug. 2021. doi:10.1109/cog52621.2021.9619019

[4] J. Junior and C. Campelo, “League of Legends: Real-time result prediction,” Anais do XVI Congresso Brasileiro de Inteligência Computacional, pp. 1–8, Dec. 2023. doi:10.21528/cbic2023-161

[5] A. L. C. Silva, G. L. Pappa, and L. Chaimowicz, “Continuous outcome prediction of league of legends competitive matches using recurrent neural networks,” in SBC-Proceedings of SBCGames, 2018, pp. 2179– 2259

[6] V. J. Hodge, S. M. Devlin, N. J. Sephton, F. O. Block, P. I. Cowling, and A. Drachen, “Win prediction in multi-player esports: Live professional match prediction,” IEEE Transactions on Games, 2019.

[7] “StandardScaler, MinMaxScaler and RobustScaler techniques – ML,” GeeksforGeeks, https://www.geeksforgeeks.org/standardscaler-minmaxscaler-and-robustscaler-techniques-ml/ (accessed Dec. 8, 2024).

[8] D. Stathakis, “How many hidden layers and nodes?,” International Journal of Remote Sensing, vol. 30, no. 8, pp. 2133–2147, Apr. 2009. doi:10.1080/01431160802549278

[9] “Auc Roc Curve in machine learning,” GeeksforGeeks, https://www.geeksforgeeks.org/auc-roc-curve/ (accessed Dec. 13, 2024).

[10] “4.2. permutation feature importance,” scikit, https://scikit-learn.org/1.5/modules/permutation_importance.html (accessed Dec. 14, 2024).

Appendix

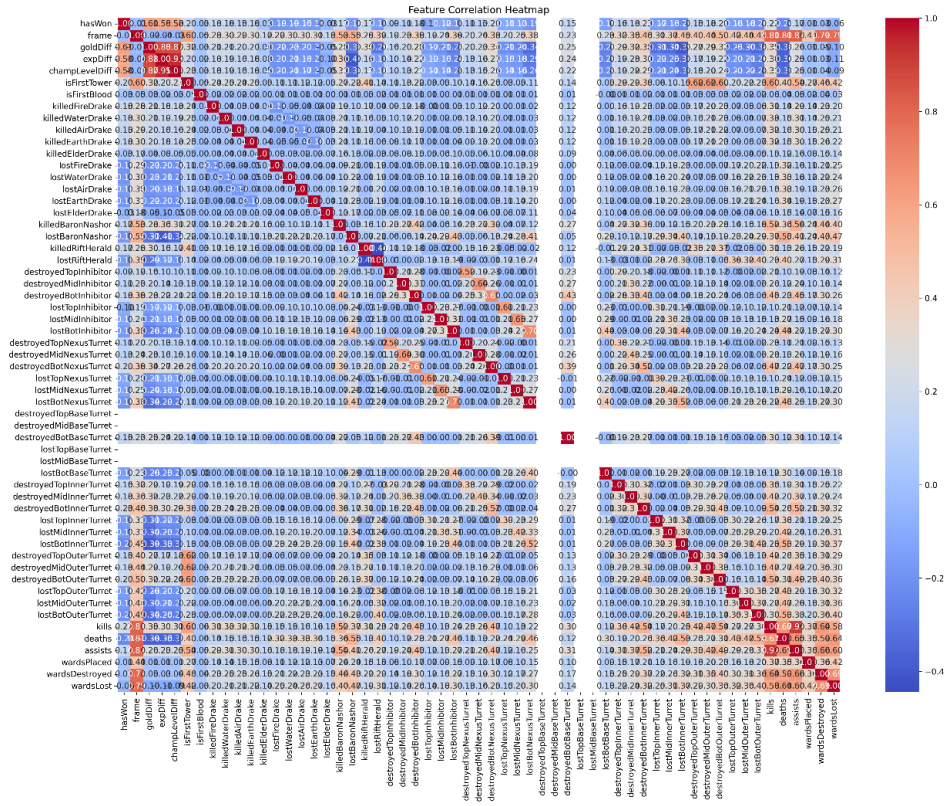


Figure 1. Correlation Heat Map of Features

```
Testing and analyzing frame group = 26, Rows: 16086
Model loaded from models/logistic_model_frame_26.pth

Results for frame group 26:
Confusion Matrix:
[[7020 1128]
 [1522 6416]]
Classification Report:
              precision    recall  f1-score   support

         0.0       0.82      0.86      0.84      8148
         1.0       0.85      0.81      0.83      7938

    accuracy                           0.84     16086
   macro avg       0.84      0.83      0.84     16086
weighted avg       0.84      0.84      0.84     16086

AUC-ROC: 0.92
Top 3 features for frame group 26:
          Feature  Importance
2   champLevelDiff    0.020059
50          deaths    0.008216
1         goldDiff    0.007436
```

Figure 2. Sample Terminal output for frame group 26 including Confusion Matrix, Classification report, AUC-ROC as well as Top 3 Features.